Robotic Navigation and Exploration

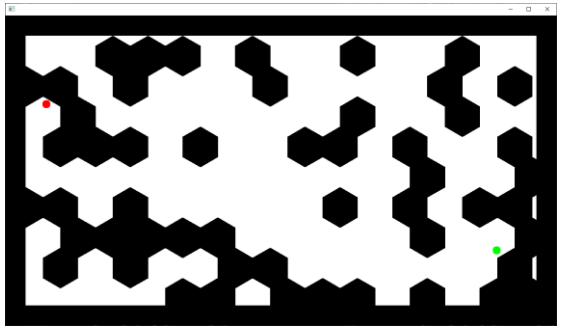
HW1: Path Planning

Min-Chun Hu anitahu@cs.nthu.edu.tw
CS, NTHU

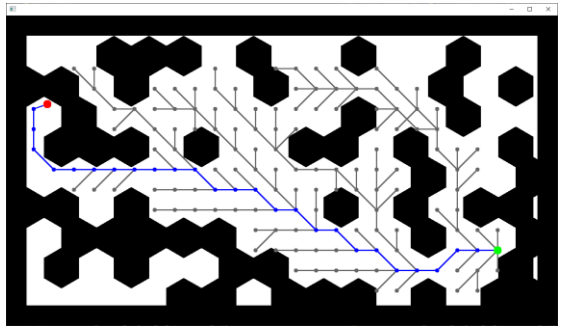
HW1 – Path Planning

- Task
  - Given a mapped environment with obstacles and start/goal pose (where pose means coordinates here), compute a valid, collision-free path
  - We provide three maps for debugging, the map for grading will be generated randomly

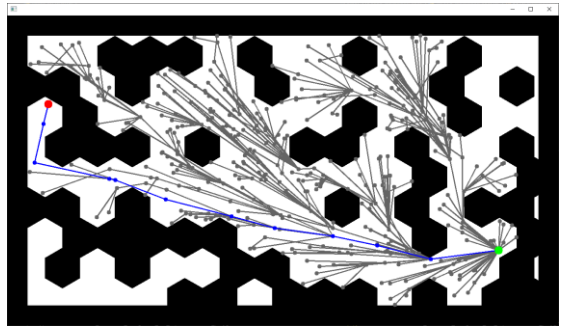
(Note that this will not be the map for grading)



Runtime generated map



A* implementation



RRT* implementation

Score & Requirement

- A* (45%)
  - Implement path planning using A* algorithm
- RRT* (45%)
  - Implement path planning using RRT* algorithm
- Report (10%)
  - Briefly explain your implementation, writing a lot won't get you more points! (to make sure you know what code you are writing)

A* Algorithm

- Cost Function: $F(v) = g(v) + h(v)$
- Consider both previous and future parameters:
  - $g(v)$: calculated path cost from start to v (Dijkstra)
  - $h(v)$: heuristic path estimation from v to target (BFS)

RRT* Algorithm

```
G.initialize(x_start)    # Initialize Graph
for i=1 to max_iter do
    x_rand ← sample()    # Sample the points in 2d space.
    x_nearest ← nearest(x_rand , G)    # Find the nearest point in graph.
    x_new ← steer(x_rand , x_nearest , step_size)    # Collision detection.
    X_near ← near_node(x_new, G, search_radius)
    x_parent ← best_parent(x_parent , X_near)    # Re-Parent
    G.add_node(x_new)
    G.add_edge(x_new , x_parent)
    G.rewire(x_new , X_near) # Rewire
    if ||x_new − x_goal|| < threshold then
        return G
```

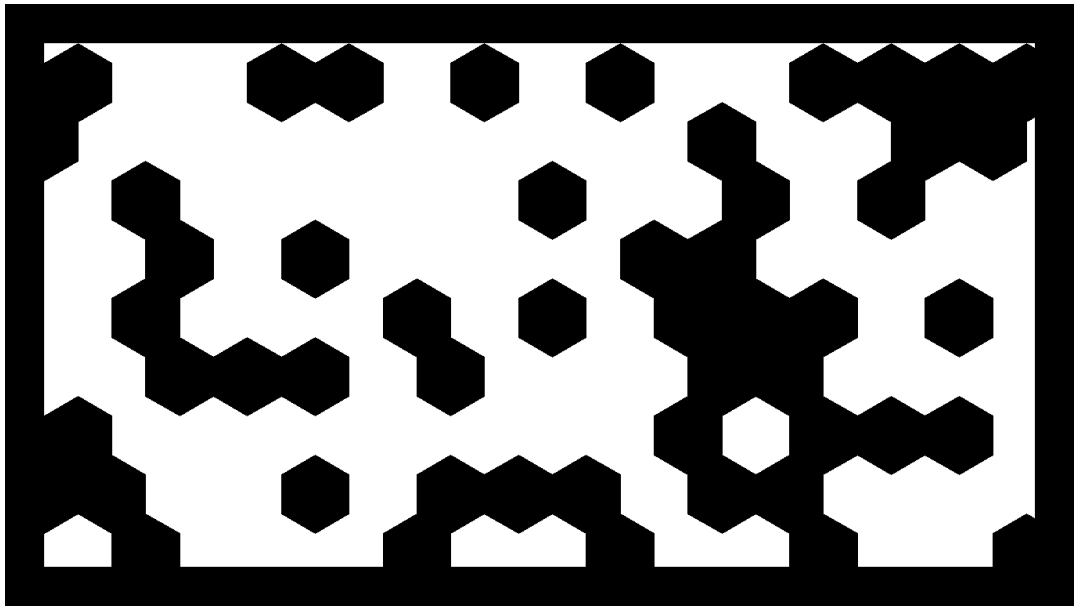
Code Structure Walkthrough

- Repository structure
  - HW1

```
|----- data
|          |----- map1
|          |          |----- map.png
|          |          |----- info.json
|          |          |----- reference_a_star.png
|          |          |----- reference_rrt_star.png
|          |----- map2
|          |----- map3
|----- path_planning
|          |----- __init__.py
|          |----- planner.py
|          |----- a_star_planner.py
|          |----- rrt_star_planner.py
|          |----- planner_utils.py
|          |----- primitives.py
|----- your_implementation
|          |----- __init__.py
|          |----- a_star_implementation.py
|          |----- rrt_star_implementation.py
|----- main.py
```

Code Structure Walkthrough

- Map data
  - map.png
  - info.json
  - reference_a_star.png/reference_rrt_star.png



```
{
    "start_coordinates": [
        1257,
        517
    ],
    "goal_coordinates": [
        140,
        433
    ]
}
```

Code Structure Walkthrough

- path_planning.primitives
  - PixelCoordinates
    - x:int
    - y:int
    - __sub__(self, other) -> PixelVector
  - PixelVector(PixelCoordinates)
    - __add__(self, other) -> PixelVector
    - __sub__(self, other) -> PixelVector
  - PathNode
    - coordinates:PixelCoordinates
    - parent:PathNode=None
    - cost:float=0

Code Structure Walkthrough

- path_planning.planner_utils
  - bresenham(x0:int, x1:int, y0:int, y1:int) -> list[tuple[int, int]]
  - world_map_to_occupancy_map(world_map:NDArray) -> NDArray
  - check_inside_map(occupancy_map:NDArray, path_node:PathNode) -> bool
  - check_collision_free(occupancy_map:NDArray, source_node:PathNode, target_node:PathNode) -> bool
  - calculate_node_distance(source_node:PathNode, target_node:PathNode) -> float
  - collect_path(goal_node:PathNode) -> list[PathNode]
  - visualize_start_goal(canvas:NDArray, start_node:PathNode, goal_node:PathNode) -> NDArray
  - visualize_visited_nodes(canvas:NDArray, visited_nodes:Iterable[PathNode]) -> NDArray
  - visualize_path(canvas:NDArray, path:list[PathNode]) -> NDArray

Code Structure Walkthrough

- path_planning.planner/a_star_planner/rrt_star_planner
  - Planner
    - plan(*args, **kwargs): # template method

```
... # some initialization
self.preloop()
for _ in range(self.iteration_limit):
        self.step()
        if self.is_done.is_set(): break
path, visited_nodes = self.postloop()
return path, visited_nodes
```

  - AStarPlanner(Planner)
    - planner_specific_init(self, grid_size:int) -> None:
    - get_neighbor_nodes(self, current_node:PathNode) -> list[PathNode]
  - RRTStarPlanner(Planner)
    - planner_specific_init(self, step_size:float, search_radius:float) -> None
    - sample_random_node(self) -> PathNode

Code Structure Walkthrough

- your_implementation.a_star_implementation
  - AStarImplementation(AStarPlanner)
    - start_node:PathNode
    - goal_node:PathNode
    - world_map:NDArray # bgr
    - occupancy_map:NDArray # bool
    - goal_threshold:float
    - is_done:Event
    - grid_size:int # for sampling neighbor nodes

    # ==========
    - preloop(self) -> None
    - step(self) -> None
    - postloop(self) -> tuple[list[PathNode], set[PathNode]]

- your_implementation.rrt_star_implementation
  - RRTStarImplementation(RRTStarPlanner)
    - start_node:PathNode
    - goal_node:PathNode
    - world_map:NDArray # bgr
    - occupancy_map:NDArray # bool
    - goal_threshold:float
    - is_done:Event
    - step_size:float # for steer
    - search_radius:float # for reparent/rewire

    # ==========
    - preloop(self) -> None
    - step(self) -> None
    - postloop(self) -> tuple[list[PathNode], set[PathNode]]

Code Structure Walkthrough

- main.py

```
# initialize planner
if args.planner == "a_star":
    planner = AStarImplementation()
    planner_argument = {
        "start_coordinates":start_coordinates,
        "goal_coordinates":goal_coordinates,
        "world_map":world_map,
        "goal_threshold":50,
        "grid_size":50,
    }
elif args.planner == "rrt_star":
    planner = RRTStarImplementation()
    planner_argument = {
        "start_coordinates":start_coordinates,
        "goal_coordinates":goal_coordinates,
        "world_map":world_map,
        "goal_threshold":50,
        "step_size":50,
        "search_radius":200,
    }
else:
    raise RuntimeError()
# plan
path, vidited_nodes = planner.plan(**planner_argument)

# visualization
visualization = visualize_start_goal(
    world_map.copy(),
    PathNode(PixelCoordinates(*start_coordinates)),
    PathNode(PixelCoordinates(*goal_coordinates))
)
visualization = visualize_visited_nodes(visualization, vidited_nodes)
visualization = visualize_path(visualization, path)
cv2.imshow("", visualization)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Implementation Rules

- Implementation should be done using only following package
  - Python standard library (Python 3.12)
  - Numpy (2.2.1)
  - Opencv-python (4.10.0.84)
- You should only modify following files
  - a_star_implementation.py
  - rrt_star_implementation.py
- You must implement Planner.postloop()
  - And return path:list[PathNode], visited_nodes:set[PathNode]
- Your implementation should reflect given parameters
  - such as grid_size in A* and step_size/search_radius in RRT*
- Violating rules on this page will result in a score of zero

Reminders

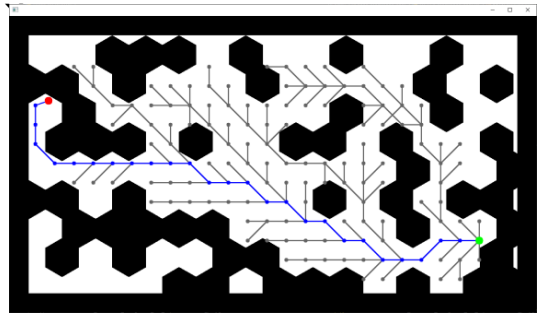
- Remember to add start_node and goal_node to visited_nodes
- Only put PathNode to visited_nodes after you actually visit the node
- PathNode.cost should be the real path cost instead of distance between nodes
- Every PathNode in visited_nodes should have a parent (except for start_node)
- PathNode with same PathNode.coordinates are considered same node (trace path_planning.AStarPlanner.get_neighbor_nodes for more details)
- random.seed(9999) is already set to ensure reproducibility
- Set Planner.is_done when distance between current_node and goal_node <= Planner.goal_threshold
- L2 distance should be used for distance calculation in A*
- RRTStarPlanner.sample_random_node should be used for sampling in RRT*

Quick Start

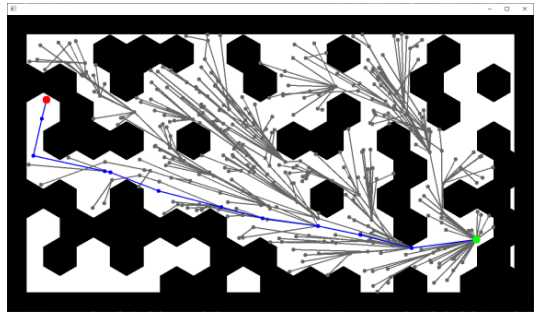
- To setup environment (using conda as example)
  - Navigate to HW1 directory
  - ```conda create –name rne_hw1 python=3.12```
  - ```conda activate rne_hw1```
  - ```pip install –r requirements.txt```
- To visualize your implementation
  - Navigate to HW1 directory
  - (activate your environment)
  - Run command

    <python/python3> main.py -p <a_star/rrt_star> -m <map1/map2/map3>

    (for example ```python main.py -p a_star -m map1```
  - You should see the visualization



A* implementation



RRT* implementation

Submission Deadline / Rules

- Deadline
  - Same as announcement on NTU COOL (2026/3/16 23:59)
  - Hard deadline, no late submission allowed
- Submission rules
  - Submission to NTU COOL should be a zipped file with following structure
    - HW1_<your_student_id>.zip (for example HW1_123456789.zip)

      |----- a_star_implementation.py

      |----- rrt_star_implementation.py

      |----- report_<your_student_id>.pdf (for example report_123456789.pdf)
    - DO NOT include any other folder/files
    - Any violation on the rules will result in a deduction of (5 points)/(each violation)

Question